Stan Bühne
Klaus Pohl

10

Domain Requirements Engineering

In this chapter you will learn:

- *The challenges of domain requirements engineering.*
- *About the interrelations between the domain requirements engineering sub-process and the product management, domain design, and application requirements engineering sub-processes.*
- *How to identify common and variable product line requirements.*
- *How to document the identified commonalities and the variability in the various requirements artefacts using the orthogonal variability model.*

In addition, a comprehensive example illustrates the definition of variability in requirements artefacts for a software product line.

10.1 Introduction

Goals of domain requirements engineering

The main goals of domain requirements engineering are the development of common and variable domain requirements and their precise documentation. Domain requirements engineering is a continuous process of proactively defining the requirements for all foreseeable applications to be developed in the software product line. A particular issue for domain requirements engineering is to identify and explicitly document the external variability. The sub-processes and artefacts closely related to the domain requirements engineering sub-process are highlighted in Fig. 10-1.

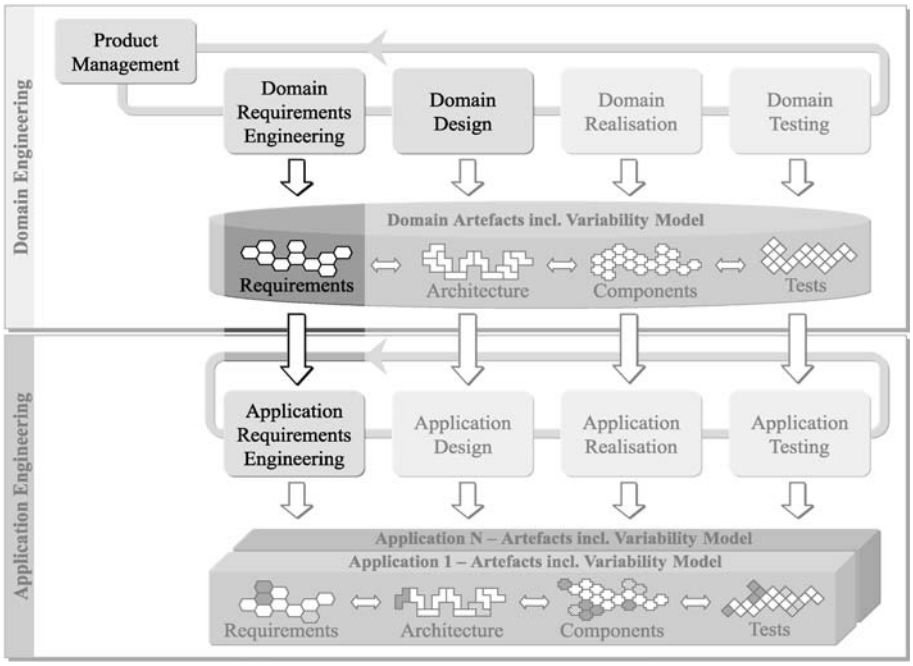


Fig. 10-1: Sub-processes and artefacts related to domain requirements engineering

Related sub-processes

Domain requirements engineering has to adhere to the specification of the product line's major features provided by product management. Based on these features, it creates detailed common and variable requirements sufficient to guide domain design (and thereby also realisation as well as testing). In addition, domain requirements engineering provides the input for the application requirements engineering sub-process, which is concerned with creating application-specific requirements artefacts.

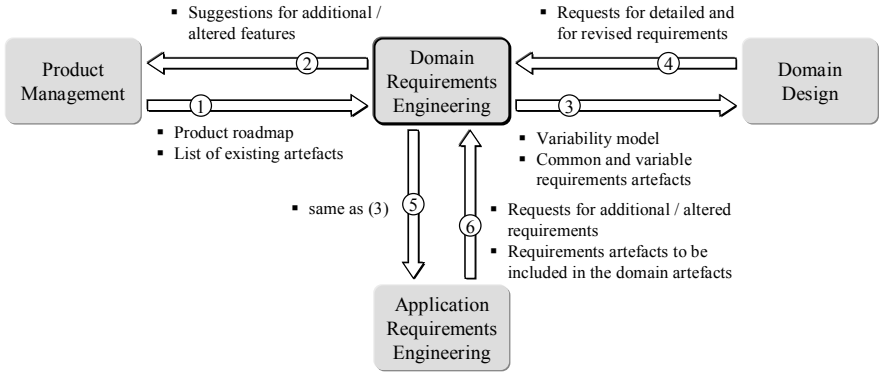


Fig. 10-2: Information flows between domain requirements engineering and other sub-processes

In the remainder of this section, we elaborate on the interrelations between the sub-processes highlighted in Fig. 10-1. The basic information flows between these sub-processes are depicted in Fig. 10-2.

Artefact flows

10.1.1 Interrelation with Product Management

Product roadmap, existing products

The key input for the domain requirements engineering process is the product roadmap from product management (first bullet ① in Fig. 10-2). This roadmap includes an initial set of intended products for the product line as well as their intended commonalities and variability. It further defines the product line with respect to the envisaged applications and a schedule for bringing out marketable products. In addition, product management identifies existing artefacts that have been developed in previous projects and which should be considered when defining the domain requirements (second bullet of ① in Fig. 10-2).

Additional/altered features

The domain requirements engineering sub-process provides suggestions on additional and altered features as well as feature refinements (② in Fig. 10-2) to product management, based on the analysis of existing products, stakeholder needs, laws, constraints, and other requirement sources.

10.1.2 Interrelation with Domain Design

Requirements and variability

The output of domain requirements engineering provided to domain design encompasses all defined domain requirements including commonality and variability as well as the definition of the product line variability in the orthogonal variability model (③ in Fig. 10-2). The variability model defines at least the external variability but may also specify part of the internal variability of the product line.

Highly interactive process

The interrelation between domain requirements engineering and domain design can be characterised by the *twin peaks model* of [Nuseibeh 2001] presented in Fig. 10-3. The figure expresses the interrelation by a spiral alternating between requirements (problem view) and architecture (solution view) that at the same time progresses from coarse to detailed. While the reference architecture increasingly takes shape, the need for more detailed requirements arises as well. In addition, existing requirements may for example turn out as too ambitious from the viewpoint of domain design, which leads to change requests (④ in Fig. 10-2).

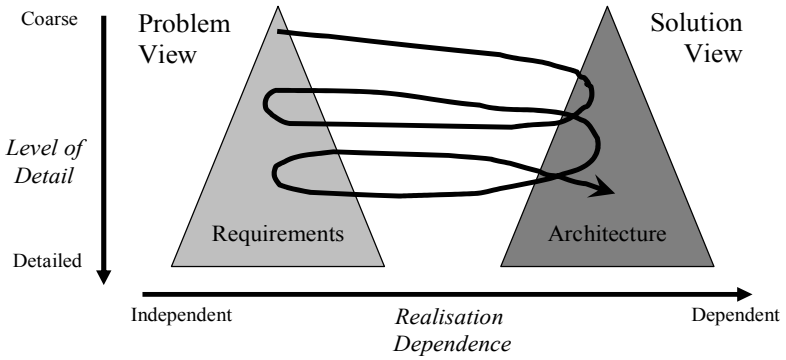


Fig. 10-3: The twin peaks model describes the interrelation of requirements and architecture (adapted from [Nuseibeh 2001])

10.1.3 Interrelation with Application Requirements Engineering

Reusable requirements

Domain requirements engineering provides the predefined common and variable requirements artefacts as well as the orthogonal variability model to application requirements engineering (⑤ in Fig. 10-2). Consequently the orthogonal variability model supports the communication of the product line variability and thus the reuse of domain requirements artefacts. We elaborate on the reuse of domain requirements artefacts in Chapter 15.

Future domain requirements

Application requirements engineering provides feedback to domain requirements engineering in terms of requests for additional and altered requirements (first bullet of ⑥ in Fig. 10-2) as well as application requirements artefacts which may be incorporated in the domain artefacts (second bullet of ⑥ in Fig. 10-2). This feedback may lead to an adaptation of the domain artefacts and thus to an evolution of the software product line. The decision on whether the feedback from application engineering is incorporated in the domain artefacts is made by product managers and other stakeholders who decide on the evolution of the product line. However, the feedback can also lead to an application-specific adaptation of the product line's capabilities, i.e. it can lead to an application-specific change. The integration of applica-

tion-specific artefacts in the domain artefacts means that domain requirements engineers, domain architects, etc., must reengineer the application artefacts and prepare them for reuse.

10.2 Traditional Requirements Engineering Activities

Before we elaborate on the details of requirements engineering for software product lines, we give a brief overview of the major requirements engineering activities in single-system engineering [Pohl 1996; Pohl 1997].

- *Elicitation*: The goal of requirements elicitation is to understand the users' needs and constraints for the system. The elicitation process uncovers needs, requirements, and constraints from different sources, such as stakeholders (e.g. customers or domain experts), user documentation, legislation, and standards. Elicitation also involves the development of new and innovative requirements for the intended product (see e.g. [Carroll 1995; Gougen and Linde 1993; Hay and v. Halle 2002; Pohl 1997; Weinberg 1988]). *User needs, requirements, constraints*
- *Documentation*: The goal of the requirements documentation activity is the well-structured recording of the elicited requirements with all necessary information. The final requirements specification is the foundation for later development phases or changes in the product [Alexander and Stevens 2003; Kovitz 1999]. To address different stakeholders such as customers and designers, it is often necessary to document requirements using different representation formats (see e.g. [Pohl 1994]). *Requirements specification*
- *Negotiation*: The goal of requirements negotiation is to achieve a sufficient consensus among different stakeholders with respect to elicited and/or documented requirements. The requirements specification is more stable during further development phases, if sufficient agreement is obtained. Without an agreement on the requirements, the project is likely to fail, for instance by running out of time (see e.g. [Wiegers 1999]). *Agreement about requirements*
- *Validation and verification*: The goal of requirements validation/verification is to prove that the system requirements are clear, complete, correct, and understandable. Validation ensures that the right requirements are documented. Verification ensures that the documented requirements are correctly defined (see e.g. [Thayer and Dorfman 1997; Sommerville and Sawyer 1997]). *Quality assurance*
- *Management*: The goal of requirements management is to maintain the requirements continuously throughout the development and system life cycle and thus to ensure that a consistent and up-to-date requirements *Continuous maintenance*

specification is available at all times. This is supported by recording trace information between requirements, from requirements to their sources, and from requirements to their realisation. It is the responsibility of requirements management to enforce adherence to the defined requirements engineering process. An up-to-date requirements specification is a prerequisite for later releases, failure fixes, and requirements reuse (see e.g. [Thayer and Dorfman 1997; Hull et al. 2002]).

The goals of the requirements engineering process can be characterised based on the three dimensions of requirements engineering [Pohl 1994]:

Understanding of system requirements

- *Specification*: This dimension characterises the achieved level of understanding about the requirements for the system under consideration. Initially, the understanding is usually weak. The goal is to achieve as complete an understanding as possible of the system requirements.

Representation of requirements

- *Representation*: This dimension deals with different kinds of representations used to document requirements. At the beginning of the requirements engineering process, typically informal representations such as sketches or natural language statements are used. The goal is to arrive at a precise requirements specification documented using as formal a requirements modelling language as possible.

Agreement about requirements

- *Agreement*: This dimension deals with the reconciliation of conflicting opinions. Typically, at the beginning of the requirements engineering process, the individual stakeholders (managers, users, domain experts, etc.) have their own view of the goals and requirements of the system. The agreement dimension characterises the gradual integration of the different views by uncovering and negotiating conflicts. The goal is to arrive at a sufficient agreement on the requirements for the system.

10.3 Challenges of Domain Requirements Engineering

Domain requirements engineering has to take into account the variability of the product line. This implies additional tasks for the requirements engineer. In this section we present the tasks that are unique to requirements engineering in software product lines.

10.3.1 Specific Activities

Precise definition of variability

The explicit documentation of the proper common and variable requirements is essential for enabling the planned reuse of requirements in application engineering. The required variability has to be documented in the orthogonal variability model in order enable its communication to other sub-processes,

such as domain design or domain testing. In this chapter, we elaborate on the three following activities:

Common requirements

- *Commonality analysis*: The goal of commonality analysis is to identify which requirements are common to all applications of the software product line (Section 10.6).

Variable requirements

- *Variability analysis*: The goal of variability analysis is to identify which requirements differ among the applications, and to determine the differences precisely (Section 10.7).

Variation points and variants

- *Variability modelling*: This activity is concerned with modelling variation points, variants, and their relationships. It is closely related to modelling variable requirements (Section 10.8).

Commonality and variability analysis

The three activities are closely related to each other. This holds in particular for the commonality and variability analysis (for additional reading on commonality and variability analysis, see e.g. [Weiss and Lai 1999]).

10.3.2 Variability in Different Views

Consistent requirements artefacts

In addition to the identification and modelling of variability, domain requirements engineering has to establish consistency across the different requirements artefacts and their documentations. The incorporation of different views facilitates the communication of the commonality and variability of the product line to different stakeholders. Examples of requirements artefacts are goals, features, scenarios, use cases, data models, behavioural models, functional models, and textual requirements (see Chapter 5 for a more detailed elaboration on the different requirements artefacts).

10.4 Overview of Major Steps

The requirements artefacts developed in domain requirements engineering encompass common and variable parts. The representation formats typically used for documenting common as well as variable requirements are explained in Chapter 5. In the following, we briefly characterise the basic steps for defining common and variable requirements.

10.4.1 Defining Common Requirements

Commonality analysis first

Before any requirements can be defined as a commonality of the product line, a commonality analysis has to be performed in order to determine which requirements are actually common to all applications; see Section 10.6. Defining common requirements consists of two basic steps:

Basic steps

1. Identify a set of common requirements.
2. Document the common requirements in detail in a representation format (features, use cases, etc.) that is suitable for the considered view.

Iteration of the steps

The steps are performed iteratively in the course of detailing and revising the domain requirements. Moreover, common and variable requirements are closely related and are therefore typically modelled together within the same artefacts.

There are at least two other important issues that have to be considered in defining common requirements:

High quality

- Common requirements are the basis for all product line applications. The work of different stakeholders depends on the quality of these requirements. Typically, much effort has to be put into keeping the requirements artefacts up to date, consistent across different views (Section 10.3.2), and to ensure a high quality (e.g. comprehensibility). Reviews help to ensure a high quality for common requirements.

Change of commonality and variability over time

- Common requirements may change to variable requirements. For example, the evolution of the product line can cause a common requirement to become variable as a consequence of introducing new variability. Documenting the rationales and assumptions on why a requirement is common helps to understand why a requirement is common and thus avoid unnecessary changes of common requirements into variable requirements. It thus prevents ending up with a fully variable, yet overly complex requirements specification.

10.4.2 Defining Variable Requirements

Modelling variability

Variable requirements are identified during variability analysis. Defining variable requirements involves modelling the variability of the product line in the variability model and documenting variable requirements in a suitable notation. Figure 10-4 illustrates the basic steps for modelling variable requirements. The four steps are:

Basic steps

1. Identify an initial set of variable requirements.
2. Develop the orthogonal variability model.
3. Document the requirements in detail in a suitable notation.
4. Relate each variable elements of the developed requirements artefact (e.g. use cases, scenarios, or classes) to the corresponding variants in the orthogonal variability model.

Steps 3 and 4 are repeated until all required views have been considered.

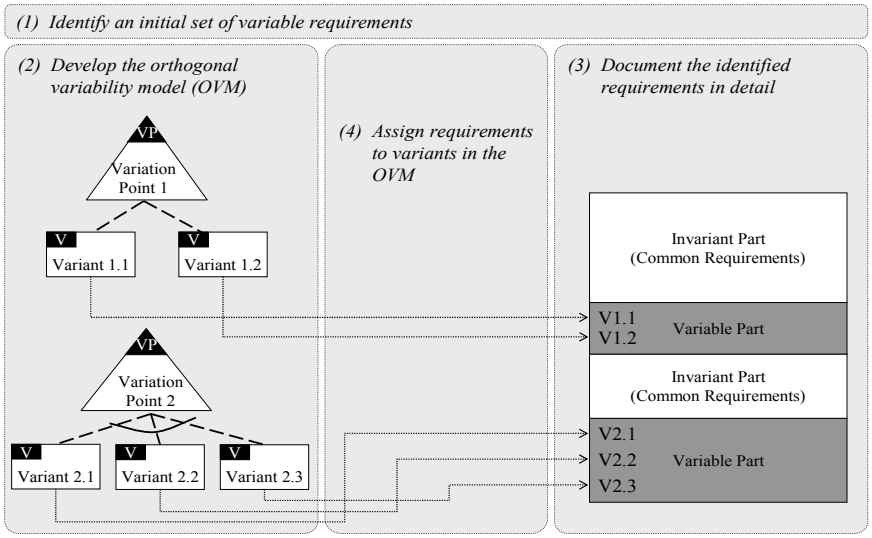


Fig. 10-4: Steps for modelling variable requirements

10.5 Requirements Sources

Stakeholders, existing products, etc.

Domain requirements engineering incorporates different *requirements sources* such as stakeholders, existing products, failure reports, or competitors' products to define the common and variable requirements for the product line. These sources are used to detail the features defined by product management.

Use of existing applications

To identify the domain requirements and their variability, domain requirements engineers can often make use of existing applications (see e.g. [Fantechi et al. 2003]). The development of a software product line rarely starts from scratch as product line engineering requires sophisticated domain experience (Section 1.4). Consequently different applications already exist that serve the markets and customer groups envisaged by the product line. These existing applications may be own applications or competitors' applications. When performing the commonality and variability analysis as explained in the following sections, requirements engineers should use, among other requirements sources, the existing applications in their domain.

10.6 Commonality Analysis

Along with the elicitation of requirements for the intended software product line applications, the commonality of the applications has to be defined.

As much commonality as possible

It is important to have as much commonality as possible, and thereby to reduce the amount of variability to the required minimum [Ardis and Weiss 1997]. Common requirements form the basis of every software product line application. The higher the amount of commonality, the less effort has to be spent in design for flexibility. Nevertheless, the amount of variable requirements should at least allow the development of individual applications that satisfy the goals and needs of the envisioned customers and/or market segments.

Exploration of applications

The identification of common requirements starts with the simultaneous exploration of the requirements for all foreseen applications of the software product line. Requirements that are identical for all these applications are good candidates for common requirements. There are different ways of identifying common requirements. A simple way to perform commonality analysis is to use an application–requirements matrix.

10.6.1 Application–Requirements Matrix

Contents of the matrix

The application–requirements matrix (see Table 10-1 for an example) gives an approximation of the commonality (and also of the variability) for a given set of software product line application requirements. The application–requirements matrix details the product roadmap, which typically defines common and variable features at a higher level of abstraction. The left column of the matrix lists the requirements of the considered applications. The applications themselves are listed in the top row. In the body of the matrix it is marked for which application a certain requirement is mandatory.

Table 10-1: Structure of an application–requirements matrix for four applications

Application Requirements	App. 1	App. 2	App. 3	App. 4
R1	mandatory	mandatory	mandatory	mandatory
R2	-	-	mandatory	mandatory
R3	-	mandatory	-	-
…	…	…	…	…

In the application–requirements matrix presented in Table 10-1 the requirement ‘R1’ is mandatory for all applications and is thus a candidate to be defined as a common product line requirement. Requirement ‘R2’ is not available in ‘App. 1’ and ‘App. 2’. Hence, it is not identified as a common requirement for the product line. The same holds for requirement ‘R3’.

10.6.2 Priority-Based Analysis

Prioritised requirements

A more sophisticated analysis of commonality can be obtained by applying the priority-based analysis scheme. Priority-based commonality analysis is based on set of requirements in which each requirement is rated by different stakeholders according to a certain scheme such as the classification proposed by Kano [Kano et al. 1996] (see Chapter 9 for more details on the Kano classification).

Basic requirements

Common requirements encompass at least the set of all *basic requirements* that every application for the envisioned domain must fulfil. Example 10-1 describes such a basic requirement.

High-priority requirements

Another indicator for commonality is a requirement having a *high priority* for a large group of customers, and the other customers do not reject it. This case is illustrated in Example 10-2.

Strategic commonality

Furthermore, it is useful to define all requirements that might be of interest to many customers in the future as common requirements. We refer to this kind of requirements as *strategic commonalities*. Strategic commonalities are foreseeable basic needs that will appear in the product line's lifetime, and thus should be implemented as commonalities to attain a stable set of common artefacts. Such kinds of commonalities may be important to differentiate from competitors' products (Example 10-3).

> ***Example 10-1:*** *Heating Control as a Basic Requirement*
>
> The Kano classification reveals that the requirement "The home automation system shall be able to control the heating of the home" is a basic requirement. Hence, it is a good candidate to be included into the set of common requirements.

> ***Example 10-2:*** *Access Control as a High-Priority Requirement*
>
> The requirement "The home automation system shall be able to control access to the home" is rated with a high priority by most customers. There are no customers who rate this requirement negatively. Thus it is likely to be accepted as a common requirement.

> ***Example 10-3:*** *Wireless Communication as a Strategic Commonality*
>
> The requirement "The home automation system shall communicate via a wireless network" is going to become a basic need in the near future. Hence, it is defined as a commonality of the product line.

10.6.3 Checklist-Based Analysis

Requirements categories

A more general approach than the priority-based identification of common requirements is the use of checklists. Each item on the checklist represents a category of requirements that should be considered as candidates for common requirements. The basic needs, high-priority requirements, and strategic commonalities described in Section 10.6.2 are examples of such categories. In addition the following general categories should be considered:

- Requirements that are prescribed by national or international laws and standards.
- Requirements that are prescribed by organisational standards.
- Requirements that only differ marginally.
- Requirements that do not conflict with each other.
- Requirements that are necessary for the technical support, like error handling, maintenance, communication, etc.

10.7 Variability Analysis

Defining variation points and variants

The goal of variability analysis is to identify requirements variability and to define the variation points and their variants related to these requirements. In software product line engineering, there is no strict need to harmonise different requirements that for example originate from contrasting customer needs or from the necessity to support different legacy systems. Rather, requirements that differ from each other indicate a need to introduce variation points and variants. However, not for every difference a variation point is defined. Whether a variation point should be introduced needs careful consideration by the stakeholders involved, as the variation point may, for instance, have significant influence on the reference architecture. The following example illustrates the introduction of a variation point due to different customer needs:

> ***Example 10-4:*** *Variability in the Home Security System due to Different Customer Needs*
>
> The different requirements of customers with regard to the security system lead to the introduction of a variation point ‘home security by’ with the variants ‘motion detection’ and ‘camera surveillance’. Each application of the product line can be customised to provide either motion detection or camera surveillance.

Examination of high-level requirements

To extract the necessary variability information, the requirements analyst has to examine the requirements for all product line applications. The identification of variability starts with the analysis of high-level requirements. The analysis reveals those requirements that are unique to a subset of the applications and those that have different characteristics in different applications. As a result of variability analysis, variation points and variants are defined.

10.7.1 Variability Analysis with the Application–Requirements Matrix

Initial set of variable requirements

Variability analysis is based on the same techniques as commonality analysis. The application–requirements matrix helps to identify variable requirements. Requirements that are only mandatory for one or a small set of applications are definitively candidates for variable requirements.

10.7.2 Priority-Based Variability Analysis

Different priorities

Requirements that have a high priority for some customers but a low priority for other customers are candidates for variable requirements. Example 10-5 illustrates the identification of a requirement with different prioritisations.

> ***Example 10-5:*** *Variability due to Different Prioritisations*
>
> A study reveals that the requirement "The system shall automatically inform the police in case of intrusion" is rated high by a significant group of customers but is of less importance for another group of customers. Hence, the requirements engineer defines this requirement as a variable requirement of the software product line.

Conflicting priorities

Similarly, requirements that are rated positively by one group of customers but are rated negatively by another group of customers can lead to the introduction of variability.

> ***Example 10-6:*** *Conflicts in Home Security Requirements*
>
> The requirement "the security system shall be equipped with a video storage system that records all surveillance video data" is appreciated by a significant group of customers. Yet the requirement "The system shall not record personal data" also has a high priority for many customers. Hence, the requirements engineer decides to define video storage as a variable requirement.

Conflicting high-priority requirements

In addition, there may be requirements with high ratings, possibly from different customer groups, which cannot be realised within the same application as they are in conflict with each other. This may be due to a real, semantic

conflict or due to technical incompatibility. This situation is illustrated in Example 10-6.

10.7.3 Checklist-Based Variability Analysis

Candidates for requirements variability

In general each difference in structure, functionality, behaviour, or quality between different applications is a candidate for requirements variability. A checklist helps in identifying requirements variability. Apart from the categories defined in Section 10.7.2, a checklist may contain, for example, the following items:

- Differences in functionality.
- Different quality attributes, e.g. with respect to safety, security, or dependability.
- Different interface requirements, in order to allow the exchange of information with different legacy systems (e.g. legacy heating control).
- Different requirements with respect to the system's user interface.
- Different design constraints, such as different databases, network types, COTS components, or operating systems.

10.8 Defining Requirements Variability

Orthogonal variability model

Defining the variability of domain requirements is a prerequisite for the sufficient understanding of, and the communication about, the variability of a product line. In Chapter 4, we introduce an orthogonal variability model for defining variability in different development artefacts. In Chapter 5, we outline how variability in various requirements artefacts should be documented using the orthogonal variability model.

Defining variability

To define the requirements variability of the indented software product line, the domain requirements engineer has to:

- Carefully define the right set of variation points and variants.
- Determine their dependencies.
- Define together with product managers which part of the product line variability is offered to the customer as external variability.

10.8.1 Variation Points and Variants

Related variability subjects

Initially it is often not clear which variant has to be related to which variation point. The documented requirements often do not state this explicitly. However, by considering the common variability subject of the variants, appropriate variation points can typically be identified. Example 10-7 illustrates this:

Example 10-7: *Finding Correlated Variants*

Application 'App. 1' has the requirement "The front door shall be secured with a keypad." whereas 'App. 2' has the requirement "The system shall provide fingerprint-based authorisation." These requirements refer to the same variability subject, namely the identification mechanism used. Hence, the variation point 'door lock identification mechanism' is defined with the two variants stated above. These variants are associated with the corresponding requirements artefacts.

Additional variants

After the variation point and the initial variants have been defined, additional variants have to be identified. Reasons for introducing additional variants can be the provision of an additional benefit for the customers or the differentiation of the software product line from competitors' products. Especially variants leading to high customer satisfaction should be taken into account in this step (we introduced the term "delighters" for requirements that lead to high customer satisfaction, see Chapter 9). The identification of additional variants can also be performed before the definition of the variation point. However, identifying additional variants after the definition of the variation point is usually better, since the variation point and the variants, which have already been identified, support the identification of additional variants.

10.8.2 Variability Dependencies

Mandatory, optional, and alternative choice

The types of the variability dependencies between a variation point and its variants and the defined alternative choices (Section 4.6) determine the permissible combinations of variants for each product line application. For some variants the appropriate variability dependency and/or alternative choice may be clear from the available requirements sources, such as product brochures (Example 10-7). If such information is not available directly, the requirements engineer has to involve the relevant stakeholders to identify the proper variability dependencies and alternative choices.

Software architects involved in variability definition

Again, by defining the variability dependencies, the requirements engineer should consider the fact that the variability can have a strong influence on the reference architecture. For example, designing a reference architecture which supports a wide range of optional variants that differ significantly in quality may be impossible. Consequently, among other stakeholders, architects should be involved in the definition of variability dependencies. Or, more generally, software architects must be involved in the definition of requirements variability.

Example 10-8: *Identifying the Proper Variability Dependency*

A product brochure for the home security system states that the system is always delivered with the feature "acoustic alarm". In addition, the customer may choose the "police information" feature, which enables the system to inform the police in the case of an attempted burglary. The variability model of the home security product line contains the variation point "alarm activation" with the mandatory variant "acoustic alarm" and the optional variant "police information".

First example application

Goals

G1:	Protection against burglary.
G2:	Catching the thief.
G...:	...
G5:	Video surveillance of the house.

Desired Features

F1:	Video surveillance.
Fn:	...
F2.2:	Inform police via phone line.

Requirements

R1:	The time between the detection of an open window and the recording of the video surveillance shall be less than 2 sec.
R2:	The alarm signal shall be deactivated by the police, by the owner, or automatically after 20 minutes.
R2.1:	The alarm signal shall start immediately after the detection of the open window or door.
R11:	The camera shall have enough storage for 5 minutes' video stream to be stored as alarm buffer.
R11.1:	The recording is only initiated if motion is detected.
R28:	The system shall be able to generate user-specific reports that document the system events.
R77:	The stored video streams of the video surveillance system shall be safe against misuse and tampering.
R78:	The time period between motion detection and start of recording shall be less than 0.5 seconds.
R78.1	The password shall consist of at least 10 characters and include special characters (such as numbers). The password shall be changed every 3 months, and an old password cannot be used again.

Fig. 10-5: Example excerpt of a requirements specification for the first application

10.8.3 Constraint Dependencies

Restricting admissible combinations

By defining constraint dependencies ("requires" and "excludes", see Section 4.6.5 for the definitions) the domain requirements engineer restricts and/or enforces the binding of variation points during application engineering. The requirements engineer has thus to determine the essential influences among the variants that exist at the requirements level. In other words, the domain requirements engineer must elicit and document the variant to variant, variant to variation point, and variation point to variation point "requires" and "excludes" dependencies stemming from domain requirements artefacts.

10.8.4 Adaptation of Product Line Variability Based on Product Management Decisions

Final decisions by product management

The final decisions on the variability in domain requirements artefacts are made by product management. In other words, product management decides:

- If a variation point identified and defined in the domain requirements engineering sub-process should be part of the product line or not, or even if a new variation point should be added.
- If the variants identified and defined in the domain requirements engineering sub-process should be part of the product line or not, or even if a new variant should be added.
- If the variability constraints and dependencies defined in the domain requirement engineering sub-process are correct, or if they have to be adapted and how.
- Whether a variation point is categorised as external product line variability or as internal product line variability.

Adaptation of requirements artefacts

Domain requirements engineers have to adapt the requirements artefacts affected by changes or adaptations of the variability definitions in the orthogonal variability model made by product management.

10.9 Example

Three example applications

In this section, we provide an example of how to identify and document variable requirements for a software product line. We first provide a simple outline of the requirements for the first three foreseeable applications. Subsequently, we demonstrate the major steps performed during commonality analysis, variability analysis, and during the documentation of common and variable requirements artefacts. Figures 10-5 to 10-7 present excerpts of the requirements for each of the three software product line applications.

Second example application

Goals

G1:	Protection against burglary in the office.
G…	...
G4:	Meet clauses of the insurance agency.
G5:	Determent of thieves through alarm signal.

Desired Features

F1:	Video surveillance.
F1.1:	Alarm activation.
F2:	Alarm signal.
F2.1:	Motion detection.
F2.4:	Automatic deactivation of alarm.
F3:	Alarm call.
F3.1:	Call by SMS (short message standard).
F3.2:	Call by internet message.

Requirements

R1:	The recording of the video surveillance system shall start no later than 1 second after the detection of the open door or window.
R2:	The police shall be informed about the burglary via radio transmission.
R3:	The alarm shall be activated in case of burglary.
R3.1:	The alarm signal shall only start if the police do not arrive within 5 minutes of alarm detection.
R3.2:	The alarm (signal) shall only be deactivated by the police or by the owner, via password and TAN (transaction number).
R10:	The time period between motion detection and start of recording shall be less than 2.5 seconds.
R13:	The camera shall have enough storage for 5 minutes' video stream to be stored as alarm buffer.
R68:	The stored video streams of the video surveillance system shall be safe against misuse and tampering.
R68.1:	The length of the password shall be at least 5 characters. The password shall only be changed if necessary.

Fig. 10-6: Example excerpt of a requirements specification for the second application

10.9.1 Commonality Analysis

Requirements matrix

The identification of common requirements is done top down from abstract requirements artefacts to detailed ones. We use the application–requirements matrix to document in which applications the requirements are mandatory. In Fig. 10-8 a set of common requirements is identified in the application–

requirements matrix. Those requirements that are mandatory for all applications constitute the commonality of the product line.

Detailed analysis

In a next step, the matrix is analysed for further commonalities between the applications. The requirements are analysed, for example, with regard to:

- The same content, represented in different words.
- Requirements that will become common in the near future.
- Requirements that only differ slightly from each other.
- Requirements that should be common from a strategic point of view.

Third example application

Goals

G1:	Safety against thieves.
G…:	...
G5:	Video surveillance of the flat.

Desired Features

F1:	Video surveillance.
F1.1:	Activation by motion detection.
F...:	…
F2.2:	Door and window sensor.
F2.3:	Manual deactivation of alarm (by owner, or police).

Requirements

R1:	The police shall be informed immediately after the detection of an open window or door.
R1.1:	The police shall be informed via internet message or SMS.
R2:	The alarm signal shall start immediately after the detection of the open window or door.
R2.1:	The alarm signal shall be deactivated by the police, by the owner, or automatically after 20 minutes.
R9:	The video surveillance shall be active as soon as activated by the user.
R9.1:	A recording is only initiated if motion is detected.
R9.2:	The camera shall have enough storage for 5 minutes' video stream to be stored as alarm buffer.
R10:	The time period between motion detection and start of recording shall be less than 0.5 seconds.
R11:	The video recording shall continue for 2 minutes after the last motion was detected.
R79:	The stored video streams of the video surveillance system shall be safe against misuse and tampering.

Fig. 10-7: Example excerpt of a requirements specification for the third application

Application–requirements matrix

Requirements	*App. 1*	*App. 2*	*App. 3*
The police shall be informed immediately after the detection of an open window or door.			mandatory
The police shall be informed via internet message or SMS			mandatory
The alarm signal shall start immediately after the detection of the open window or door	mandatory		mandatory
The alarm signal shall only start if the police does not arrive within 5 minutes after alarm detection		mandatory	
The alarm signal shall be deactivated by the police, by the owner, or automatically after 20 minutes	mandatory		mandatory
The video surveillance is active as soon as activated by the user			mandatory
…	…	…	…
A recording is only initiated if a motion is detected.	mandatory		mandatory
The camera shall have storage for 5 minutes' video stream to be stored as alarm buffer	mandatory	mandatory	mandatory
The time period between motion detection and start of recording shall be less than 0.5 seconds	mandatory		mandatory
The time period between motion detection and start of recording shall be less than 2.5 seconds		mandatory	
The video recording shall continue 2 minutes after the last motion was detected			mandatory
The stored video streams of video surveillance system shall be safe against misuse and tampering	mandatory	mandatory	mandatory

Fig. 10-8: Using the application requirements-matrix to identify variability

Example 10-9: *Identifying Similar Goals*

Application 1, G1: “protection against burglary”

Application 2, G1: “protection against burglary in the office”

Application 3, G1: “protection against thieves”

The three goals of the different applications can be described by one common goal: “protect against burglary”.

In Example 10-9 we illustrate three different goal descriptions that can be summarised by a single goal "protection against burglary".

10.9.2 Variability Analysis

Identifying variable requirements

The application–requirements matrix presented in Fig. 10-8 is used to identify an initial set of variable aspects among the requirements of different applications. The candidates for variable requirements, which we focus on in the following, are highlighted in Fig. 10-8 by grey bars. The requirement "the alarm signal shall be deactivated…" is variable because it is only mandatory for applications "App. 1" and "App. 3". In addition, the requirement itself contains variable aspects; see Examples 10-10 and 10-11.

Example 10-10: *Variable Aspects Within a Requirement*

The requirement differentiates among the following variants: 'The alarm signal shall be …'

Variant 1: "…deactivated by the police"

Variant 2: "…deactivated by the owner"

Variant 3: "…deactivated automatically after 20 min."

Example 10-11: *Variants Among Different Applications*

For the authentication variant 'password authentication' the security requirements differ among the applications. App. 1 requires a high password quality, whereas App. 2 requires a low password quality.

Variant 1 'high password quality': Req-78.1, App. 1: "The password shall consist of at least 10 characters and include special characters (such as numbers). The password shall be changed every 3 months, and an old password cannot be used again."

Variant 2 'low password quality': Req-68.1, App. 2: "The password shall be at least 5 characters long. The password shall only be changed if necessary."

10.9.3 Defining Variation Points and Variants

Determining the variability subject

The correct definition of variation points is essential, since a variation point provides the central location for binding the variability during application engineering. Typically, the variation subject is a good indicator for a variation point and its name. In Example 10-12, we define the variability subject by abstracting from related variants.

Example 10-12: *Defining a Variation Point for a Set of Variants*

By abstracting from the requirements "the alarm signal shall be deactivated…" we define the variation point 'alarm deactivation'. Moreover, we define only two variants as manual deactivation can be done by any person authenticated by the system.

Hence, for the variation point 'alarm deactivation' the following variants are available:

Variant 1: "…manual deactivation by an authenticated person"

Variant 2: "…automatic deactivation after 20 minutes"

10.9.4 Defining Variability Dependencies

After determining the variation point and its variants, the required variability dependencies have to be defined. We illustrate this step in Fig. 10-9 for the variation point 'alarm deactivation' from Example 10-12.

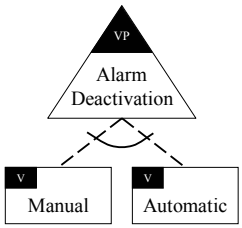


Fig. 10-9: Example of a variation point with a an alternative choice of two variants

Example 10-13: *Definition of Variability Dependencies*

The variants 'manual' and 'automatic' shall be optional variants for deactivating the alarm. Exactly one of them must be selected for an application. Hence, we relate them to the variation point 'alarm deactivation' using optional variability dependencies and group the dependencies by an alternative choice with a range of [1..1]. As this is the default range we do not show it in Fig. 10-9.

10.9.5 Defining Constraint Dependencies

Variant to variation point constraint

In Example 10-14, we illustrate the use of a constraint dependency between a variant and a variation point. If the variant at the source end of the constraint dependency ('video surveillance') is selected, the variation point at the target end ('video surveillance quality') has to be bound by selecting the desired variants. The selection thus has to take into account the defined variability dependencies and alternative choices.

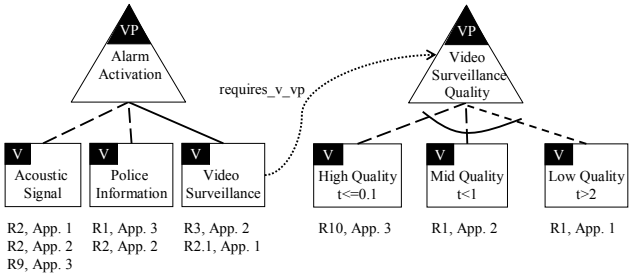


Fig. 10-10: "Requires" dependency between a variant and a variation point

Example 10-14: *Use of the "Requires" Constraint Dependency*

The 'requires_v_vp' dependency between the variant 'video surveillance' and the variation point 'video surveillance quality' (Fig. 10-10) means that the video surveillance system is available in different quality variants, such as required by the individual applications.

10.9.6 Documenting Domain Requirements

Variants in a use case scenario

Commonality and variability have to be defined in the various requirements artefacts and related to the corresponding concepts (variants and/or variation points) in the orthogonal variability model. Figure 10-11 illustrates an excerpt of a variable use case scenario, which is enriched by XML tags. These XML tags structure the textual scenario in different elements. Due to the definition of the elements in the scenario the variable parts (e.g. steps, preconditions, etc.) of the use case scenario can be related to the corresponding variants in the variability model.

10.10 Differences from Single-System Engineering

Identifying and defining commonality and variability

The main goal of domain requirements engineering is the prospective development of common and variable requirements artefacts for the product line in order to enable large-scale reuse in application engineering. Consequently, the requirements engineering activities (elicitation, documentation, negotiation, and validation/verification) do not deal with a single application but with the requirements of all envisioned product line applications. The requirements engineer has to involve a potentially large number of different stakeholders (product managers, architects, customer groups, maintenance staff, etc.) and different requirements sources (legacy systems, country laws, etc.) to be able to identify all relevant common and variable requirements.

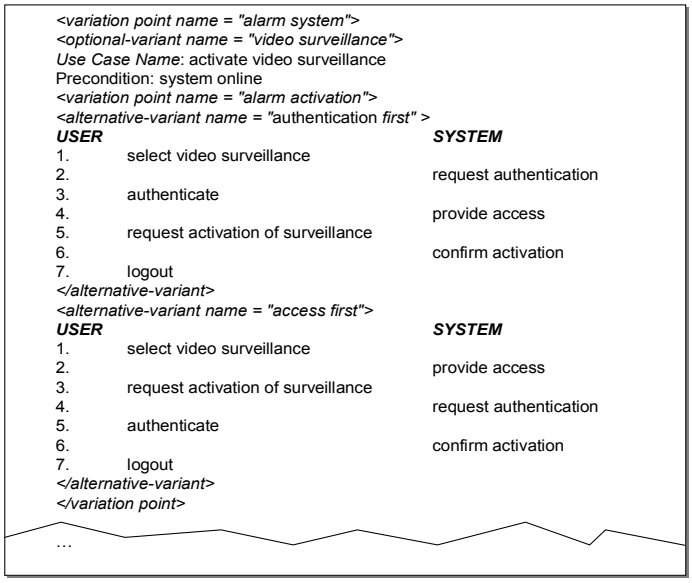


Fig. 10-11: Example of a variable, textual scenario description

10.11 Summary

Identification of commonality and variability

Along with the elicitation of requirements from different sources, the domain requirements engineering sub-process has to identify which requirements are common to all applications, and which requirements differ among the applications. Hence, also during domain requirements engineering a commonality and variability analysis is performed. The application–requirements matrix provides a synopsis of the high-level requirements for several applications and can thus be used to support the identification of commonality and variability. A more sophisticated analysis can be performed on a set of prioritised requirements. In addition, checklists can be used to guide the identification of common and variable requirements.

Definition of variability

To support efficient communication and to enforce consistency of the variability of the software product line, the variability is defined in the orthogonal variability model. Variability modelling involves the identification and definition of variation points, variants, variability dependencies, and constraint dependencies. Variation points and variants are identified by abstracting from variable requirements and/or by grouping similar requirements artefacts. Architects have to be involved in the definition of product line variability as the variability has a strong influence on the reference architecture. External variability is defined together with product management.